

Molly Pate

11 April 2026

EN605.617

Module 11 Reflection

Goals

Despite not feeling super creative with the mathematical operations for this assignment, my goal was to perform a simple transformation on (x,y) pairs, which were represented as OpenCL vectors, while leveraging buffers and sub-buffers to fine tune what kernel execution was accessing specific data, experimented with MapBuffers to mapping GPU memory directly into the host's address space and reduce latency, vector math, as well as becoming more familiar with contexts, and playing with multiple queues. I also experimented with timing different thread and block sizes and looked into timing techniques to simply to check that my implementation matched my intuition that parallelism is a great benefit, especially as the work grows in size.

Triumphs

One of my biggest successes was implementing sub-buffers alongside memory mapping. Instead of relying on traditional buffer copies, I used `clEnqueueMapBuffer` to enable a zero-copy approach, which reduces overhead and is especially effective on unified memory systems. I also added device-aware alignment by querying `CL_DEVICE_MEM_BASE_ADDR_ALIGN`, ensuring that sub-buffer offsets were always valid for the target hardware. This made the implementation more robust and portable across devices.

Another major success was integrating kernel profiling to analyze performance behavior. Using `clGetEventProfilingInfo`, I was able to measure execution time across different workloads and work-group sizes. For example, increasing the global size (total threads) from 1,024 to over 4 million only increased execution time from about 0.012 ms to 0.138 ms. I also observed the impact of work-group size: using a local size of 1 resulted in much slower execution (~1.51 ms) compared to 64 or 256 (~0.04 ms), demonstrating how proper work-group configuration significantly improves GPU utilization.

Challenges

One of the main challenges was implementing sub-buffers correctly. I originally aimed to split work between the CPU and GPU, but on macOS I ran into issues where CPU queue synchronization was unreliable. To solve this, I switched to a two-stage GPU approach (one queue), splitting the workload in half and using a global offset so the second half writes into a sub-buffer without overwriting the first.

Another challenge was handling differences between OpenCL versions across platforms. Since macOS only supports older OpenCL features, I would consistently run into compiler warnings about deprecation issues when swapping over to my EC2 instance.